Lab report with scope instructions

Share

projectinstruuctions.docx DOCX

COSC_455_Project_1_Design_Notes_Illustrat…ded (1).docx DOCX

tests(1) (1).zip ZIP


PDF

use std::env; use std::fs; // // ========== ==== Compiler Trait ==== ========== // /// Compiler trait pub trait Compiler { fn compile(&… PASTED

I attachedd a lab also in insdturcitons it says stay in scope I have providedd I think all u need follow precisely

Ran 6 commands, read a file, shared files

I built the compiler as a single `main.rs` with a `Cargo.toml` (package name `lolcompiler`, no crates). I ran it on all 10 of your test files and the results are below. The `cargo test` unit tests I added also pass.

Test	Result
Test1.txt	Usage error because it isn't a `.lol` file, and no HTML is written
Test2, 3	Valid HTML; Test3 handles the lowercase `#hai` and `#kbye`
Test4	Lexical error on line 3: `'#HEAD' is not a recognized annotation`, no HTML
Test5	Syntax error on line 6: `unexpected '#MKAY' inside the document body`, no HTML
Test6	`42` is substituted into the paragraph
Test7	Static semantic error on line 9 for the undefined `answer`, no HTML
Test8	The first paragraph says `42` and the second says `happiness`, so scoping works
Test9	Two variables resolve correctly
Test10	Produces `<a href="https://www.youtube.com/watch?v=aboZctrHfK8">…</a>`

How it follows the instructions

- **Traits:** it uses the `Compiler`, `LexicalAnalyzer` and `SyntaxAnalyzer` (Option 1) traits exactly as given. I did not use the `Result` version.
- **Lexer:** it works character by character and finishes before parsing, as in your lab. Annotations and keywords are stored in upper case, so the rest of the code ignores case.

- **Parser:** it is recursive descent with one function per nonterminal, built from the BNF PDF. It records every accepted token in order, as the "parse tree".
- **Scope and HTML:** a scope stack walks that record. A block pushes a scope, `#IHAZ` adds a definition, `#GIMMEH VAR` searches from the innermost scope outward, and a block end drops its definitions. The HTML is built in memory and written only if every stage succeeds.
- **Driver:** it requires exactly one argument ending in `.lol`, writes the `.html` next to the input, and opens it with `open` on macOS or `start` on Windows.

Differences from the BNF PDF and spec

- **Bold with a variable:** the PDF's `<bold>` allows only plain text, but your project text shows `#GIMMEH BOLD #GIMMEH VAR x #OIC #OIC`. I allowed variable use inside bold. The grammar comment in the code shows the change, so update your Phase 1 BNF to match.
- **Two-token peek:** `#MAEK` and `#GIMMEH` each start several productions (HEAD, PARAGRAF and LIST for `#MAEK`). The parser therefore peeks at the keyword after them. The PDF grammar needs this too, so mention it if you're asked about "one-token lookahead".
- `get_char`: the trait comment says it should terminate at end of input, but I followed your lab and return `'\0'`.

Before you submit

- **Comments:** the project says the code comments must be your own words. I wrote them as a starting point, so rewrite them in your own words until you could explain every function in a follow-up question.
- **AI disclosure:** include the AI/LLM disclosure in `readme.txt` and put the transcript in `design/`.
- **Struct name:** my struct is `LolCompiler`. If your instructor's unit tests construct a specific name, check what it expects.
- **Mac check:** I couldn't test the browser launch here, so run one test on your Mac or Windows machine.

Files are in `lolcompiler/`:

- `src/main.rs`
- `Cargo.toml`

↓ Download all

remove commentsd

Ran a command, shared files ▸

I removed every comment from `main.rs`, including the doc comments and the grammar block. After that it still builds, the unit tests pass, and Test2, 4, 5, 7 and 8 give the same results as before.

The project requires well-commented code (Phase 5 and the 5 code-quality points). You'll need to add your own comments before you submit.

verify u kept in labs scope

Listing which Rust features the lab permits ▸

I can't fully verify that. The only evidence I have of what your class covered is the lab file and the project text, and my code goes beyond the lab in several places.

Matches the lab

- Traits and `impl ... for`, structs with `new()`, and `Vec<String>` / `Vec<char>`.
- `loop` / `while` / `break`, `if` / `else if`, and `std::mem::take`.
- `.iter().any(...)`, `.reverse()`, `.pop()`, `.clone()`, and `unwrap_or_default()`.
- `eprintln!`, `std::process::exit`, `env::args`, and `fs::read_to_string` with `unwrap_or_else`.

Not in the lab

- `match`:
  - The lab uses only `if` / `else`.
  - I use `match` in `translate()` and `unexpected()`.
  - The patterns include `"#MKAY" | "#OIC"`, a binding arm (`text =>`), and `let (open, close) = match ...`.
- `if let Some(line) = ...` in `next_token()`.
- Tuples: `Vec<(String, String)>` for variables, plus `for (n, v) in vars.iter().rev()`.
- Associated functions without `self`: `Self::emit` and `Self::resolve`.

- **Methods the lab doesn't use:** `.cloned()`, `.last()`, `.truncate()`, `.join()`, `.repeat()`, `.trim()`, `.starts_with()`, `.ends_with()`, `.to_uppercase()`, `.map()` with a closure, and `format!`.
- **Browser opening:** `std::process::Command` and `cfg!(target_os = ...)`. The project text points you to this, but the lab doesn't use it.
- `#[cfg(test)]` **module:** `assert_eq!` tests, which the project text mentions but the lab doesn't show.

I don't know whether class covered the items in the last four groups. The project text does say to stay within the BNF, ANTLR and Rust features seen in class or lab, so I'd treat the `match` blocks and tuples as the riskiest.

I can rewrite the code to use only the lab's constructs: `if`/`else` chains instead of `match`, two parallel `Vec<String>`s instead of tuples, and so on. The behavior would stay the same, and I'd re-run all 10 tests. Want me to do that?

```
    Self {
            input:
source.chars().collect(),
position: 0,
line: 1,
current_build: String::new(),
build_line: 1,
tokens: Vec::new(),
token_lines: Vec::new(),
annotations: ann.iter().map(|s|
s.to_string()).collect(),
keywords: kw.iter().map(|s|
s.to_string()).collect(),
}
} this looks out of scope'\
```

Show more

Ran 2 commands, shared files ▸

You're right. The `ann.iter().map(|s| s.to_string()).collect()` lines used a closure inside `.map()`, and the lab doesn't use that. I replaced them with the lab's own style, `vec!["#HAI".into(), ...]`, the same way the lab builds its word lists.

I rebuilt it, and the unit tests and Test2, 4, 5, 6, 7, 8, 9 and 10 give the same results as before. The updated `main.rs` is attached.

Other things in the file are still outside the lab, as I listed earlier: `match`, tuples, `if let`, `Self::emit` and `Self::resolve` as functions without `self`, `format!`, `Command`, and `#[cfg(test)]`. The `.any(|x| ...)` closures are fine, since the lab uses those. If you want, I can rewrite the rest of it in the lab's style.

ok if I want to ensure im in the constraints what do i do send u all the labs idk

Yes, send me the labs. I only have the one lab file (the Lolspeak one), so I've been guessing what class covered. Without the others I can't verify the scope, so the more you give me, the better the check.

What to send

- **Labs 1-5:** the `.rs` files or the lab PDFs/docx. The project says its grammar and parser follow Labs #1-5, so these are the best evidence of what counts as in scope.
- **Lecture slides or notes on Rust:** especially anything on `match`, enums, tuples, closures, `Option` / `Result`, and `std::process::Command`. Those are my main doubts.
- **Anything the instructor posted about allowed features:** an announcement, a syllabus note, or a Piazza/Blackboard post.

What I'll do with them

1. List every Rust feature the labs use.
2. Compare `main.rs` against that list and flag anything not covered.
3. Rewrite the flagged parts using only what the labs show, then re-run all 10 tests.

What you can do yourself

- **Check the code you can't explain.** The project says you must fully understand everything you submit. If you can't explain a line, that's a sign to simplify it.
- **Ask your instructor:** "Are `match`, tuples, and `std::process::Command` okay for Project 1?" The project text already points you to a "Rust without crates" guide for opening the browser, so `Command` is probably fine.
- **Use the lab as the reference.** If a construct doesn't appear in the labs and you can write the same thing with `if` / `else`, loops, `Vec`, and `String`, use the simpler version.

Upload what you have and I'll check it.

the labs dont rlly help I can tell u the chapters of rust modules I lweasened thog

Yes, that works. Tell me which Rust chapters or modules you covered. The Rust Book chapter numbers or titles are best, if that's what your class followed.

For reference, these are the chapters that match the features I'm unsure about:

- `match` and enums: Rust Book ch. 6 ("Enums and Pattern Matching").
- Tuples: ch. 3.2 ("Data Types").
- `if let`: ch. 6.3.
- Closures: ch. 13.1. The `.any(|c| ...)` calls need this. Your lab has similar ones, but it's worth confirming.
- Methods and associated functions (`Self::emit`): ch. 5.3.
- Traits: ch. 10.2, which the lab and project already use.
- Tests (`#[cfg(test)]`, `assert_eq!`): ch. 11.
- `std::process`, `env`, `fs`: ch. 12 ("An I/O Project"), which covers `env::args` and `fs::read_to_string`. `std::process::Command` isn't taught in the Book, but the project links a guide that uses it.

Once I know what you covered, I'll compare it with `main.rs`, rewrite anything outside it, and re-run all 10 tests. If you also remember anything specific from lecture, such as "we never did closures" or "we did match", add that.

Data types, cionpoound data types, functions, program flow control, ownership, references, structs, traits, lifelines

Ran 3 commands, shared files ▸

I rewrote `main.rs` to use only the topics you listed. It builds with no warnings, and all 10 tests and the unit tests give the same results as before.

What I changed

- `match` (enums and pattern matching) is now `if` / `else if` chains in `translate()` and `unexpected()`.
- `if let` is gone. `next_token` uses `unwrap_or`, and the file write in `main` uses `unwrap_or_else` like your lab's file read.
- Iterator chains (`.last().cloned()`, `.iter().rev()`, `.join()`, `.repeat()`) are replaced with an index check and plain `while` loops.
- `vars.truncate()` is now a `while` loop that pops.
- `Self::emit` and `Self::resolve` are plain functions outside the `impl`.
- The constructor uses the lab's `vec![".."..into()]` form, from the earlier fix.

What stays, and why

- **Tuples:** `Vec<(String, String)>` for variable name and value. Compound data types are on your list.
- **Structs, traits, references,** `&mut`**, ownership via** `.clone()`**:** all covered.
- **Closures:** `.any(|a| *a == up)` in `lookup` and `.any(|c| c.is_whitespace())` in `expect_word`. Your lab uses the same pattern, but it's the one construct your list doesn't name. I can swap them for loops if you want.
- `std::process::Command` **and** `cfg!`**:** needed to open the browser, and the project text points you to a guide for it.
- `#[cfg(test)]` **unit tests:** optional. The project mentions `assert_eq` tests but class may not have covered them, so you can delete that module without affecting anything else.

The updated `main.rs` is attached.

also why do u wrote each test as a function r u sure thats what my professdor wantrs

Checking which deliverables the project actually requires ▸

No, I'm not sure, and the project doesn't ask for them. I added those tests myself to check the code, and they aren't a requirement.

What the project asks for

- **Phase 4 (testing):** `.lol` input files in a `test` directory. That means your own test inputs plus the instructor's Test1-10, and the self-grading form from the zip.
- **Grading:** 20 points are for handling his test cases and for "Rust assert_eq unit tests I will apply to the required, trait functions". He writes and runs those unit tests, not you. They will probably call `compile`, `next_token`, `current_token`, `get_char`, `lookup` and similar, on your struct.

So the `#[cfg(test)]` module in my `main.rs` isn't required, and class may not have covered it. Delete it unless you want the extra check.

Where I'm unsure

- **Struct name:** his tests need to create your compiler struct. I named mine `LolCompiler`, and I don't know what he expects. Your lab used `LolspeakCompiler` with `::new()`, so renaming to match that is a reasonable guess. Check the project text or ask him.

- `get_char` at end of input: the trait comment says it should terminate the program, but I return `'\0'` like your lab. His `assert_eq` test could depend on this, so check how the lab's version behaved.

I can delete the test module and rename the struct if you tell me which name to use. I'd re-run all 10 tests after.

r u sure by my own testgs yhe didnt just mean ttry out the code with different inputs

Confirming that the test inputs are .lol files, not Rust unit tests ▸

You're right, and that matches what I said. "Your own tests" means `.lol` input files that you run through the compiler, not Rust test functions. The project text says so directly: you need tests that use each construct, check variable scope, and include some illegal input.

The Rust test functions in my file are separate. I added them as a quick check, and they're optional. The only Rust tests the project mentions are the `assert_eq` ones your instructor applies himself.

What goes in your `test` directory

- His Test1-10 files.
- A few of your own `.lol` files. They should cover:
  - a list with several items
  - `#NEWLINE`
  - a link inside a paragraph
  - a variable used inside a list item or bold
  - a variable defined inside a list item, so it goes out of scope after the item
  - nested scope with three levels
  - a missing `#KBYE`
  - a nested paragraph
  - a list item outside a list
  - a second `#MAEK HEAD`
  - an undefined variable used after its scope has ended
- The self-grading form from the zip.

If you want, I can write those `.lol` files and run them through the compiler. I'd tell you which ones should fail and why, and you'd check that the results match.

Just reread directions again plz and do what ut hink is right

Ran 3 commands, shared files ▸

I reread the directions and made three changes.

1. Removed the Rust test functions from `main.rs`. The directions say your instructor applies the `assert_eq` unit tests himself. Your own tests are `.lol` input files, so the test functions were extra code outside what the class covered. Everything else in the code is unchanged.

2. Wrote 24 test files in `test/mine/`, next to your instructor's Test1-10, as the Phase 4 `test` directory asks. I ran each one:

- **6 valid files** (`ok_*`) all compile. They cover:
  - lists, `#NEWLINE` and links
  - variables inside bold and list items
  - shadowing across three scope levels, with the outer value restored afterward
  - comments and mixed-case annotations with no head
- **18 invalid files** (`err_*`) all fail with the right error type and produce no `.html`:
  - 13 are syntax errors, such as a missing `#KBYE`, a nested paragraph, a second head, text after `#KBYE`, and a variable definition that isn't first in its block
  - 3 are lexical errors
  - 3 are semantic errors, where a variable is used after its scope ends, leaks out of a list item, or is defined only in a later sibling block
- **Your instructor's tests:** Test1 is rejected as a non-`.lol` file, Test4 gives a lexical error, Test5 a syntax error and Test7 a semantic error, with no HTML for any of them. The rest compile.

3. Added a `readme.txt` template with blanks for your GitHub link, which OS you built on, and 2-3 sentences on how you used AI. The directions require all three.

Still left for you

- **Build and test on your own machine:** run `cargo build --release` on your Mac or Windows machine, put the binary in `bin/`, and re-run the tests there. I couldn't test the browser launch.
- **Comments:** the code has none now, and the directions require your own comments.
- **Remaining submission items:** the `design/` folder (BNF, ANTLR file, state diagram, AI transcript), the GitHub repo with frequent commits, and the filled-in self-grading form.
- **Struct name:** my struct is `LolCompiler`. Your instructor's unit tests may expect a different name, so check with him.

Download all

what module is the unwrap or else thing from

Confirming where the standard library's Result and Option methods are used

`unwrap_or_else` isn't from a module you import. It's a method on the standard library's `Result` and `Option` types, which are in the Rust prelude (automatically in scope).

In my code it's called on `fs::read_to_string(filename)` and `fs::write(...)` in `main`. Both return a `Result` (an `io::Result`, which is a `Result` whose error type is `std::io::Error`):

- If the `Result` is `Ok(value)`, it returns the value.
- If it is `Err(err)`, it runs the closure you give it. Mine prints the error and calls `std::process::exit(1)`.

That's the same pattern your lab uses for reading the input file:

```rust
let sentence = fs::read_to_string(filename).unwrap_or_else(|err| {
    eprintln!("Error reading file '{}': {}", filename, err);
    std::process::exit(1);
});
```

The `unwrap_or(...)` and `unwrap_or_default()` calls are the same idea on `Option` (and `Result`). In my code they handle `pop()` on an empty vector: you get a default value (an empty string, or 0) instead of a failure.

The `use std::fs;` at the top is only for `fs::read_to_string` and `fs::write`. The `unwrap_or_*` methods need no import.

out of the topics I told u earlier which would vectors be under

Sorting Vec among the compound data types ▸

Vectors fit best under compound data types, but I'm not certain your class put them there.

In the Rust Book, tuples and arrays are the compound types (ch. 3), and `Vec` is a collection from the standard library (ch. 8). Many courses teach vectors right after arrays, which is probably what your "compound data types" covered. Vectors also come up under ownership and references, because pushing, popping and borrowing a `Vec` all depend on those rules.

Either way, your lab uses `Vec<char>` and `Vec<String>` for the input and the tokens, and the project's `SyntaxAnalyzer` trait is described as working "over Vec<String>". The project itself relies on vectors, so they're safe to use.

`String` is also a collection type, so it is in the same position as `Vec`.

so this new code passes all tests?

Confirming test results in the sandbox.

Yes, for the tests I could run. After the last change I re-ran everything on the final `main.rs`:

- **Your instructor's Test1-10:** all behave as the descriptions say.
  - **Test1.txt** is rejected as a non-`.lol` file.
  - **Test4** gives a lexical error, **Test5** a syntax error and **Test7** a semantic error, and none of them produces HTML.
  - **The rest** (Test2, 3, 6, 8, 9 and 10) compile to HTML with the variables resolved as expected. For example, Test8 gives `42` in the first paragraph and `happiness` in the second.
- **My 24 extra tests:** all 6 valid ones compile, and all 18 invalid ones fail with the right error type and no `.html` file.

What this doesn't prove

- **My checking:** I judged the HTML by reading it, not with his grading. I have not seen how he scores "correct output".
- **Hidden tests:** the project says his grading tests are "likely variations" of the examples, so some inputs I haven't seen could behave differently.
- **His `assert_eq` unit tests:** these depend on names and signatures I can't see, such as the struct name `LolCompiler`.
- **Testing platform:** all of this ran on Linux. The automatic browser launch only failed here because the sandbox has no browser, so it still needs one run on your Mac or Windows machine.

Build it on your machine and run the 10 tests there before you submit.

is this code easy for me to add my own comments to u think or no

Mostly yes. The file is easy to comment because it follows the same order as the project's pieces. It has a few harder spots, which I list below.

Why it's easy

- **Parser:** each `parse_*` function matches one rule in your BNF. You can comment each one by pasting its rule above it and saying in a line what it checks.
- **Lexer:** it follows the character-by-character design from the design notes. `tokenize` is the main loop, and `read_annotation` and `read_keyword` are the two states.
- **Small helpers:** `expect`, `consume`, `is_text`, `peek_token` and `syntax_error` are short and do one job.
- **The traits:** you can comment each method from its description in the project text.

Where you'll need more time

- `translate()`: this is the densest part. Three parallel structures stay in sync as you walk the recorded tokens:
  - `vars` holds the visible definitions.
  - `marks` stores how long `vars` was when each block started, so leaving a block can drop that block's definitions.
  - `closers` holds the closing tag for each open block.

  The `i += 2`, `i += 4` and `i += 5` jumps skip a fixed number of tokens for each construct. If you can explain why a variable defined in a paragraph disappears at `#MKAY`, you understand the whole function.
- `peek_token()`: know why it's there. `#MAEK` and `#GIMMEH` each start several rules, so the parser looks one token further to choose.
- `record` and `record_lines`: these are the "parse tree". The parser saves every accepted token, and `translate()` reads that list afterward.
- `resolve()`: it searches `vars` from the end backward, which is what makes the nearest definition win.

How to do it

1. Start with the traits, then `tokenize`, then the parser, then `translate()` last.
2. Run it on Test8 and follow one variable through by hand. That makes the `translate()` comments much easier.
3. If you can't explain a line, ask me about it or simplify it.

I can walk through `translate()` line by line, or explain any other function, if you want.

i mean he said AI is allowed I just gotta commentr piece by piece I guess

Replying briefly, reminding them comments must be their own. ▸

That's the right way to do it. AI use is allowed, but the project says the comments must be yours and show that you understand the code, so explaining each piece to yourself first and then writing the comment is the correct approach.

A workflow that works

1. Pick one function.
2. Ask me to explain it, or work it out yourself by running a test file through it.
3. Write the comment in your own words, as if explaining it to a classmate.
4. Move to the next function.

A good order, smallest pieces first

1. The three traits at the top (use the project text's descriptions).
2. The lexer: `get_char`, `add_char`, `lookup`, then `read_annotation`, `read_keyword`, and `tokenize`.
3. The parser helpers: `expect`, `consume`, `is_text`, `peek_token`.
4. The `parse_*` functions, in the order of your BNF.
5. `translate()`, `resolve()` and `emit()`.
6. `main`.

If you want to start now, tell me which function and I'll explain what it does and why it's written that way. I won't write the comment for you, but if you draft one I can tell you whether it matches what the code does.

does it say to commit it to my github I made previously or new one

Checking whether the repository requirement covers existing code. ▸

It doesn't say. The project text only requires that you keep your source code on GitHub with frequent, informative commits, and that you give a link to a public repository of your code in the readme.

A new repository for this project is the safer choice. Part of the grade (5 points) is repository quality: the commits and their messages. A separate repo keeps your history clean for the grader. If you reuse an older one, make sure the project sits in its own folder and the commit history shows this project's work.

Whichever you choose, the repo must be public, because the project says that is the only way your instructor can help you or grade it. If you're unsure which to use, ask him.

Commits

- **Frequent commits:** don't commit everything once at the end. A history like this reads well:
  1. Project setup
  2. Lexer
  3. Parser
  4. Scope resolution and HTML output
  5. Tests
  6. Comments, added function by function
- **Commit messages:** the project asks for informative ones, for example "Add lexer: tokenize annotations and plain text".

Link in the readme: put the repo link in `readme.txt` where the template has `<PASTE YOUR PUBLIC REPO LINK HERE>`.

only 5 points is the github ...

Reading the grading breakdown for the code assignment.
Not quite, and I should have been clearer. The 5 points are only for repository quality (commits and commit messages). GitHub counts in other places too:

- **Another 20 points:** the grading text says 20 points are "for code development following tasks 1-4 **and configuration management use**". Configuration management means version control, so GitHub use feeds into that score as well.
- **Required to be graded:** your public repo link has to be in the submission. The project says that's how your instructor grades the code and helps with technical questions, and it says projects not following the submission format won't be graded.
- **Visible history:** the repo also backs up your claim that you wrote and understood the code. The commit history shows the work built up piece by piece, which matters given the AI-use rules.

So it's worth doing properly. Make a public repo, commit in small steps with clear messages, and add the link to `readme.txt`. The commits for adding your own comments one function at a time fit this well.

ye so if the assignment is due tmr at midnight and the ocde is now done how do i send frequent gits

Planning honest commits over the coming day

Make real commits from now until the deadline, and don't fake an earlier history. Backdating commits or building a history that implies you worked over weeks would misrepresent your work, and the project says your instructor reads this repo, so it could cost you more than a thin history would. Your commits can honestly show the work you do from now on: adding your own comments piece by piece, running tests, and building on your machine.

Setup (about 5 minutes)

1. Create a new **public** repo on GitHub with no README, so the first push has no conflicts.
2. In your project folder (the one with `Cargo.toml`), run:

```
git init
git branch -M main
echo "target/" > .gitignore
git add .gitignore Cargo.toml
git commit -m "Set up Rust project for lolcompiler"
git remote add origin <your repo URL>
git push -u origin main
```

Commits that match real work

```
git add src/main.rs
git commit -m "Add lexer, parser, scope resolution and HTML output"
git add test/
git commit -m "Add instructor tests and my own valid/invalid .lol tests"
git add readme.txt
git commit -m "Add readme template"
git push
```

Then commit after every function or group of functions you comment, for example:

```
git commit -am "Comment lexer: tokenize, read_annotation, read_keyword"
git commit -am "Comment parser helpers: expect, consume, peek_token"
git commit -am "Comment parse_* functions"
git commit -am "Comment translate and resolve"
git commit -am "Comment main and browser opening"
```

Run `git push` every few commits, so GitHub always matches your computer.

Other things to do tonight

- Build on your Mac or Windows machine, put the binary in `bin/`, and commit it with a message like "Add release build for macOS".
- Run all the tests on that machine. If you fix anything, commit it with a clear message.
- Commit the readme with your repo link, the OS and your AI-use sentences filled in.
- Commit the `design/` folder files (BNF, ANTLR, diagram, AI transcript).

Write each commit message about what you actually changed. Doing the comment pass function by function gives you a natural stream of commits.

ui think like 3 or 4 commits is good

Weighing how many commits the request calls for.

3 or 4 is the minimum I'd accept. The project asks for "frequent" commits, and the 5 repository points are graded on commits and commit messages, so a handful of big commits will probably score less than a longer history. It's your call, but more commits cost you very little tonight.

You'll make several commits just by commenting the code function by function, because each group of comments is a natural commit. If you do these, you'll have about 8-10 real commits without any extra work:

1. Project setup (`Cargo.toml`, `.gitignore`)
2. Source code (`main.rs`)
3. Test files
4. Comments on the lexer
5. Comments on the parser
6. Comments on `translate` and `main`
7. Readme with your link, OS and AI-use sentences
8. Design files and the build for your OS

If you really only have time for 3 or 4, make them these, with clear messages:

1. Set up the project and add the source code
2. Add the tests
3. Add comments to the code
4. Add the readme, design files and build

The thing that matters more than the count is that every message says what the commit actually does.

ok walk me thru how to get github setup in vs code again with new repository

Deciding how to present the steps.

Install Git and sign in

Check you have Git by opening the VS Code terminal (Ctrl+` or Cmd+`) and running `git --version`. If it's missing, install it from git-scm.com and restart VS Code. Then click the Accounts icon (bottom-left person icon) and choose Sign in with GitHub, or make an account at github.com first.

1 2 3 4 5 6 7 View all steps Next

The steps above cover the VS Code setup. A few things people get wrong:

- **Folder level:** open the folder that directly contains `Cargo.toml`. If you open a parent folder, the repo will include extra files.
- **Publish Branch:** if the button doesn't appear, you haven't made a first commit yet.
- **Sign-in:** if publishing fails, sign in again from the Accounts icon and retry.
- **Terminal alternative:** you can create an empty public repo on github.com (no README), then run `git remote add origin <URL>` and `git push -u origin main`.

If you hit an error message at any step, paste it here and I'll help you fix it.

OK i got my reposotiry in vs code now how do I get the rust file to ruin

Setting up Rust in VS Code.
To run it you need Rust installed, then you run it from the VS Code terminal.

1. Check that Rust is installed
Open the terminal in VS Code (Ctrl+ `or Cmd+`) and run:

```
cargo --version
```

If it says the command isn't found, install Rust from rustup.rs (on Windows it may also ask for the Visual Studio C++ build tools, so accept that). Then close and reopen VS Code and try again.

2. Check you're in the right folder
The terminal should be in the folder that contains `Cargo.toml`. Run `ls` (Mac) or `dir` (Windows) to confirm you see `Cargo.toml` and `src`.

3. Run it on a test file

```
cargo run -- test/Test2.lol
```

The `--` separates cargo's options from your program's argument. The first run compiles the code. It should print `Compiled ... -> ...Test2.html` and open the page in your browser.

4. Try the error cases

```
cargo run -- test/Test4.lol
cargo run -- test/Test7.lol
cargo run -- test/Test1.txt
```

These should show a lexical error, a semantic error and a usage error, with no `.html` file created.

5. Build the executable for submission

```
cargo build --release
```

The finished program is in `target/release/`. It's named `lolcompiler.exe` on Windows and `lolcompiler` on Mac. Copy it into a `bin/` folder, and test it directly:

```
bin/lolcompiler test/Test2.lol
```

On Windows use `bin\lolcompiler.exe test\Test2.lol`.

Optional: install the rust-analyzer extension in VS Code to get error highlighting and a Run button above `main`.

If you get an error, paste the message here and I'll help you fix it.

no ok I got cargo --version tell me how to get rudset setup then imma paste file into main

Settling the binary's name inside the existing repo folder

Since your repo folder is already open in VS Code, set up the Rust project right inside it.

1. Create the project in your repo folder

Open the terminal in VS Code (Ctrl+ `or Cmd+`). Make sure it's in your repo folder, then run:

```
cargo init --name lolcompiler
```

- `cargo init` sets up a Rust project in the current folder.
- `--name lolcompiler` makes the executable come out as `lolcompiler`. The project requires that name.

It creates `Cargo.toml`, `src/main.rs` and a `.gitignore` that already ignores `target/`. It won't make a new git repo, because your folder already has one.

2. Paste in the code

Open `src/main.rs`. It has a small "Hello, world!" program. Select all of it, delete it, and paste in my `main.rs`.

3. Check `Cargo.toml`

It should look like this:

```
[package]
name = "lolcompiler"
version = "0.1.0"
edition = "2021"

[dependencies]
```

If `edition` says 2024 or something else, it's fine to leave it.

4. Add the tests

Copy the `test` folder (with the instructor's Test files and my `mine` folder) into the same folder as `Cargo.toml`.

5. Run it

```
cargo run -- test/Test2.lol
```

It should compile and open the page in your browser.

If the folder you opened isn't empty and `cargo init` complains, paste the message here.

where do I put the tests

Choosing where the test folder goes.

Put them in a folder named `test` in your project's root, in the same folder as `Cargo.toml`:

```
your-repo/
├── Cargo.toml
├── src/
│   └── main.rs
├── test/
│   ├── Test1.txt
│   ├── Test2.lol
│   ├── ...
│   ├── Test10.lol
│   ├── Test_Case_Descriptions.txt
│   ├── (the self-grading rubric .docx)
│   └── mine/
│       └── (my 24 .lol files)
├── bin/
├── design/
└── readme.txt
```

- **Folder name:** the project says the folder must be called `test`. Your instructor's zip unpacks to `tests`, so rename it to `test` (singular).
- **What goes in it:** his files, my `mine` folder, and the rubric form once you've filled it in.
- **Commands:** from the root, `cargo run -- test/Test2.lol` runs a test, and `cargo run -- test/mine/ok_01_list_newline_link.lol` runs one of mine.
- **Output location:** the `.html` file is written next to the input file, so the results appear inside `test/`.
- **Git:** you can commit the `.lol` files but not the generated `.html` files. To keep those out, add `*.html` to your `.gitignore`.

I got src target folder then gitignkolre cargo.lco mand cargo.toml

That's the correct layout. `cargo init` created these:

- `src/`, which holds `main.rs`
- `Cargo.toml`, the project file
- `Cargo.lock`, which Cargo generates and keeps up to date. Commit it, since this is a program and not a library.
- `.gitignore`, which already ignores `target/`
- `target/`, the build output. It's ignored, so it won't go into your repo.

What's left

1. **Paste the code:** open `src/main.rs`, select all, delete, and paste in my `main.rs`.
2. **Add the tests:** put the `test` folder (his files, plus my `mine` folder) next to `Cargo.toml`.
3. **Check `.gitignore`:** it should contain `target/`. Add a second line, `*.html`, so the generated HTML doesn't get committed.
4. **Run it:** `cargo run -- test/Test2.lol`.
5. **Commit:** stage everything except `target/` (which `.gitignore` already skips) and commit with a message like "Set up project and add source code".

If you want to commit in smaller steps, commit the project setup first, then `main.rs`, then the tests.

what was it supposed to do

Assuming you mean the compiler, here's what it should do when you run it.

What it does

1. It reads a LOLCODE Markdown file (`.lol`), for example one containing `#HAI`, `#MAEK PARAGRAF`, `#GIMMEH BOLD` and `#KBYE`.
2. It checks the file in three stages: the lexer reads the characters into tokens, the parser checks the structure against the grammar, and the scope check resolves each variable to its value.
3. If everything is valid, it writes an `.html` file next to the input and opens it in your browser.

4. If anything is wrong, it prints one error message and creates no HTML file.

What you should see when you run it
`cargo run -- test/Test2.lol` prints `Compiled 'test/Test2.lol' -> 'test/Test2.html'`, creates `test/Test2.html`, and opens a page that says "This is test case 2." with the title "Test 2" in the browser tab.

For the error cases, `cargo run -- test/Test4.lol` should print the lexical error `'#HEAD' is not a recognized annotation` and create no file.

If you meant something else, such as the `.gitignore` or the commit step, tell me which one.

yes I see that

Good, that means the code is working on your machine. Next:

1. **Run all the tests.** Run `cargo run -- test/Test1.txt` through `Test10.lol` and check each against its description in `Test_Case_Descriptions.txt`. The error tests should print their message and create no `.html` file.
2. **Run a few of mine** from `test/mine/` to check the scope and error cases.
3. **Commit and push** what you have now.
4. **Build the executable** with `cargo build --release`. Copy `lolcompiler` (or `lolcompiler.exe` on Windows) from `target/release/` into a `bin/` folder, and test it by running `bin/lolcompiler test/Test2.lol`.
5. **Fill in** `readme.txt` with your repo link, the OS you built on, and your AI-use sentences.
6. **Add your own comments** to `main.rs`, one function at a time, committing after each group.
7. **Finish the design files** (BNF, ANTLR file, lexer state diagram, AI transcript) if you haven't already, and fill in the self-grading rubric form.

Tell me if any test doesn't behave the way its description says.

wait so from directions in my AI use file if I jujst be homnest and show my prompts and I used vibe coding will I be penalized

Checking the assignment rules against the implementation requirements. ▸

Going by the directions, honest disclosure with your transcripts should not be penalized on its own, but being allowed to use AI doesn't excuse missing the other requirements. I can't say how your instructor will grade it, so I'm only reading what the directions say.

What the directions say

- AI use, including vibe coding, "is allowed (and somewhat encouraged)".
- The final submission "must strictly" do all of these, and projects that don't "will not be graded":
  1. **Requirements:** meet the implementation requirements. Yours does: character-by-character lexer, recursive descent parser, and the provided traits.
  2. **Comments:** have comments you wrote yourself, not AI-generated ones.
  3. **Class features:** stay within the BNF, ANTLR and Rust features seen in class or lab.
  4. **Understanding:** be code you fully understand, in case he asks follow-up questions in lab, projects, exams or elsewhere.
  5. **Transcripts:** include the AI transcripts or permalinks in `design/`.
- `readme.txt` needs 2-3 sentences on whether and how you used AI.

What this means for you

- Being honest and including the transcripts covers items 5 and the readme.
- The risk is in items 2 to 4. Writing the comments yourself and knowing how each function works is what protects you. If he asks you to explain `translate()` or why the parser peeks a token ahead, you need to be able to answer.
- On item 3, I removed the constructs I was most worried about, but I don't know exactly what your class covered. If you're unsure about `Command` for opening the browser or the `.any(|x| ...)` closures, you could ask him.

What to do tonight

- Include the full transcript of this conversation in `design/`, or permalinks to it.
- In the readme, say plainly what you did. For example, write that AI generated the first version of the compiler, that you tested it on the instructor's tests and your own, and that you added the comments.
- Make sure that last part is true by the time you submit.

opk if I do comments n stuff and I tweaked to b e more in line with my requirements i should b e fine

Probably, yes, as long as three things are true when you submit.

1. **The comments are yours.** Write them yourself in your own words, not AI text with a few words changed.
2. **You understand the code.** The directions say you must be able to explain it in follow-up questions. Before you submit, make sure you can explain these:
   - how `translate()` handles scope
   - why `peek_token()` exists
   - how the lexer turns characters into tokens
3. **Your readme matches what you did.** Only write that you "tweaked" or adapted it to fit the class's requirements if you really did. Make that part true by tonight, and say it plainly in the readme.

Two other things can still cost you points no matter how good the comments are:

- **Missing pieces:** the `design/` folder (BNF, ANTLR, lexer state diagram), the transcript, the filled-in rubric form, `bin/`, and the repo link in `readme.txt`. Not following the submission format means the project won't be graded.
- **Features outside class:** if you use something you can't explain or that your class never covered, that's the part that could get flagged. If you're unsure about `Command` or the closures, ask your instructor.

With those done and the 10 tests passing on your machine, you've followed what the directions require. I can't promise how he'll grade it.

did u use the lab4 skeleton code heavilyh?

Adapting the earlier lab code's structure for the new one ▸

I used it as a guide, but I only ever saw one lab file, the Lolspeak one, so I can't confirm it's Lab 4. It was attached to your first message without a lab number.

Taken from it

- **Overall shape:** the `Compiler` and `LexicalAnalyzer` traits, a `SimpleLexicalAnalyzer` struct, and a compiler struct that holds the lexer and `current_tok`.
- **Tokenizing up front:** it tokenizes the whole input, stores the tokens in reverse, and pops them one at a time.
- **Lexer methods:** the `get_char` / `add_char` / `lookup` shape, including `get_char` returning `'\0'` at the end of the input.
- **Parsing style:** one function per rule, helper checks, `eprintln!` and `std::process::exit(1)` for errors.
- **Driver:** `main` reads the file with `unwrap_or_else`.
- **Constructor:** the `vec!["...".into()]` word lists.

Written for this project, not from the lab

- **Lexer:** the `#` annotation and keyword states, and line-number tracking.
- **Parser:** the grammar, the two-token peek, and the `record` of accepted tokens.
- **Back end:** the scope stack, `translate()`, HTML output, the browser launch, and the `.lol` check.

So the structure leans on the lab, but most of the code is specific to the LOLCODE project. If your instructor compares the two, he'll see the same layout and a lot of new logic. Describe it that way in the readme, and don't say you wrote it without the lab or without AI.

Ok the lolpseka was it helpgful

Yes, it helped a lot. It showed me how your class structures a compiler, so the code uses the same pieces instead of a design you'd never seen.

Where it helped most

- **Trait names and signatures:** the lab showed the traits being implemented on structs, and the project's traits use the same names and method shapes.
- **Tokenize up front, then pop:** this fits the `next_token` and `current_token` design in your project text.
- **Error handling:** printing with `eprintln!` and exiting is how your class handles errors.
- **Staying in scope:** it was my best evidence of which Rust features your class uses. That's why I changed the constructor to the lab's `vec![... .into()]` style and kept the `unwrap_or_else` file read.

Where it fell short

- It's a sentence grammar, so it never needed annotations, keywords, nesting, variables or scope. All of that is new code.
- It reads whitespace-separated words, so the lexer for `#` annotations and plain-text runs is new too.
- It's one lab, so it didn't tell me everything your class covered. That's why I asked you for the chapters.

It's good evidence of your class's style, but it only reaches so far. For the rest you need to understand the new parts yourself.

also my prof wants like AI messagers right

Explaining how to pull out AI conversation transcripts ▸

Yes. The directions say that if you used AI, you provide "a file (or files) with the AI/LLM transcript text, or permalinks to these interactions" in the `design` directory. The AI/LLM Use section repeats it: the submission must contain the transcripts or permalinks used to aid in design and development.

What to include

- **This conversation,** since the compiler came from it.
- **Any other AI chats** you used for the project, such as for the BNF, the ANTLR grammar or the state diagram. Include every tool, not just Claude.

How to get them

- **Permalink:** if your chat app has a share option for a conversation, use it to make a link and paste it into a text file in `design/`. Check that the link opens when you're logged out. I can't make the link for you from here.
- **Text file:** if you can't make a link, copy the whole conversation into a `.txt` or `.md` file in `design/`, for example `ai_transcript_claude.txt`. Include your prompts and the replies, not just the code.

Also needed

- The 2-3 sentences in `readme.txt` saying whether and how you used AI.
- The transcript files committed to your repo, so the repo matches your submission.

Don't edit or trim the transcript to hide parts of how you got the code. Your instructor expects to see it, and the directions treat missing or misleading disclosure as a reason not to grade the project.

should I put all teasts inside now or not

Yes, put them all in now, because the project wants them in the `test` directory, and the sooner they're in the repo the sooner you can run them on your machine.

What goes in `test/`

- Your instructor's files: `Test1.txt`, `Test2.lol` to `Test10.lol`, and `Test_Case_Descriptions.txt`.
- My `mine/` folder with the 24 extra tests. The project says the folder should ideally have more than just his files.

- The self-grading rubric `.docx`. Leave it blank for now and fill it in after you've run the tests.

Steps

1. Copy everything into a folder called `test`, not `tests`, next to `Cargo.toml`.
2. Make sure `.gitignore` has `*.html`, so the generated pages don't get committed.
3. Run the 10 instructor tests and a few from `mine/`, and compare each result with its description.
4. Commit with a message like "Add instructor tests and my own valid and invalid test files", then push.

If you add or change any tests later, commit those separately.

also is this vibe coding?

Explaining vibe coding's definition with its Wikipedia link.

Mostly yes. Vibe coding means describing what you want to an AI and letting it write the code, with the person guiding and testing rather than writing it line by line. That's how most of this compiler was made. I wrote it from your project files, and you steered the scope changes and the testing.

Your instructor's directions allow this, and say it "should be well within your abilities to complete without its use". Describing it honestly matters more than which label you use. In the readme and transcript, say that AI generated the first working version, that you directed the changes to fit your class's scope, and that you tested and commented it yourself.

The label won't protect you in a follow-up question. If he asks you to explain a function, you need to be able to, so work through the parts you can't explain yet before you submit.

how come this project is way more advanced then skeleton coxdce

Because the two programs solve very different problems. The skeleton is a warm-up, and the project is the full compiler it was meant to prepare you for.

The Lolspeak skeleton

- It checks one fixed sentence shape: noun phrase, optional adverb, verb, noun phrase.

- Every token is a single word from a fixed list, so the lexer only splits on whitespace.
- It has no nesting, no variables and no scope.
- It produces no output file. It just prints whether the sentence is valid.

Your project

- **Lexing:** it has annotations that begin with `#`, keywords that follow them, and runs of plain text, so the lexer needs real states.
- **Nesting:** paragraphs, lists, items and bold sit inside one another, which needs a much bigger grammar and a parser with many more functions.
- **Variables:** it needs static scoping, which means tracking definitions as blocks open and close.
- **Output:** it has to translate everything into HTML, write a file only if there were no errors, and open the browser.
- **Errors:** it must tell lexical, syntax and semantic errors apart.

The skeleton shows the pattern: traits, a lexer, and one parser function per rule. The project takes that pattern and extends it to a whole language.

That gap is also why the comments and understanding requirement matters. Reading the skeleton code is easy, but this project is a step up, so work through the lexer, parser and `translate()` until you can explain them in your own words.

ye but did project say I needesd all of that

Listing which extras went beyond the requirements ▸

Mostly yes. Almost everything in the code is something the project text requires.

Required by the project

- **Lexer:** character by character, with annotations, plain text and a lexical error for something like `#HEY`.
- **Parser:** recursive descent over a grammar with paragraphs, lists, items, bold, links, comments and variables, using the provided traits.
- **Static scope:** variable definitions, shadowing, and an error for use of an undefined variable.
- **HTML output:** written with the same name and a `.html` extension, and opened in the default browser.
- **Errors:** one message per error, with “as much error information as possible”, and no output file when any error occurs.
- **Driver:** exactly one argument, a `.lol` extension, and a usage error otherwise.

Extras I added that the project doesn't ask for

- **Error hints,** such as "a title is only allowed inside #MAEK HEAD". The line numbers are a reasonable reading of "as much error information as possible", but the hints are my addition.
- **Indented HTML.** The project says spacing doesn't need to be preserved.
- **The Linux browser fallback** (`xdg-open`). The project only mentions Mac and Windows.
- **The 24 extra test files.** These are optional, though the project says to have more than just his.
- **Variable use inside bold.** The BNF PDF doesn't allow it, but the project's own example does.

If you want less to explain, the hints, the indentation and the Linux branch are the easiest to delete without changing what the tests check. I can strip them out if you'd like a smaller file, and I'd re-run all the tests afterward.

is test files in the code?

No. The test files are separate input files and aren't part of `main.rs`.

- `src/main.rs` contains only the compiler. I removed the Rust test functions earlier, so there are no tests in it.
- `test/` holds the `.lol` and `.txt` input files, both your instructor's and my 24 extra ones. The compiler never loads them on its own. You pass one to it, for example `cargo run -- test/Test2.lol`.

That separation matches the project, which asks for a `test` directory of input files.

I think the code might be too polished cause alot of this stuff semes unecessary

Ran a command, shared files ▸

I agree, so I trimmed the extras. I removed these:

- **Error hints:** the parenthetical advice like "a title is only allowed inside #MAEK HEAD". Messages now just say what was unexpected and where, with the line number.

- **Indentation in the HTML:** `emit` now just writes a line, and `translate()` no longer tracks depth. The browser output looks the same.
- **Linux branch:** `open_in_browser` now tries Windows `start` or Mac `open`, the two systems the project names.
- **The fancier "found" message:** it just shows the current token.

The file is now about 630 lines, down from about 720.

I re-ran everything and the behavior is unchanged. Test1 is rejected, Test4, 5 and 7 fail with the right error type and no HTML, and the other six instructor tests compile. All 24 of my extra tests still pass or fail as intended. The updated `main.rs` is attached.

What's left is what the project requires, plus things you could still cut if you want it smaller:

- **Line numbers in errors:** the project asks for "as much error information as possible". I'd keep them.
- **Reverse-popped tokens and `token_lines`:** the lab's design plus line tracking. It's the simplest way to give line numbers.
- **`peek_token` and the two-token lookahead:** the BNF needs it, since `#MAEK` and `#GIMMEH` each start several rules.
- **`record` and `record_lines`:** your "parse tree" and the way `translate()` gets its input.

If you tell me which parts still feel like too much, I'll cut those too.

copuld it be less? cause I still think I dont need 609 lines

Ran a command ▸

Yes, but not by much without dropping something the project requires. The file is 629 lines, 57 of them blank.

Lines the project requires

- **The three traits** (`Compiler`, `LexicalAnalyzer`, `SyntaxAnalyzer`) are about 55 lines, copied from your instructions.
- **One `parse_*` function per rule** is in the project text (17 of them), and together they're about 200 lines. They're short, but there are many.
- **The lexer** (character by character, with annotations and text) is about 150 lines.

- `main` (usage checks, writing the file, opening the browser) is about 40 lines.

Where I could cut

- `translate()` is the biggest block, at about 100 lines. Most of it is walking the saved tokens and jumping `i += 2`, `i += 4` and so on.
  - The parser could build the HTML and track scope as it goes, which would remove `translate()`, `record` and `record_lines`. That saves about 120 lines and removes the part that's hardest to explain.
  - The cost is that scope checking and output stop being a separate stage after parsing. The project describes them as separate tasks (a semantic analyzer that takes the parse tree), so I wouldn't do it without checking with your instructor.
- **Line numbers:** removing them from errors would save about 20 lines (`token_lines`, `record_lines`, `build_line`). I'd keep them, because the project asks for "as much error information as possible".
- **Small merges:** `parse_simple_item`, `read_keyword` and a few helpers could be folded together for 20-30 lines. That would make the code denser, not simpler.

Below about 500 lines, you start dropping things the project asks for or making the code harder to read. Length isn't graded. What the project cares about is whether you can explain each piece.

I can cut the line numbers and merge a few helpers if you want it shorter, or I can leave it as is. Which would you like?

so in its current form could I comment it all myself

Yes, you can, as long as you build up to the hard parts. I'd estimate 2 to 4 hours if you actually work through each function, which is realistic for tonight.

Easy

- **The three traits** (about 55 lines): copy the meaning from the project text.
- **Short helpers:** `expect`, `consume`, `is_text`, `peek_token`, `found`, `syntax_error`, `emit`. Each does one small thing.
- **Most `parse_*` functions:** put the BNF rule above each one and add a line on what it checks.
- `main`: the usage check, the file read, the `.html` name and the browser launch. Each of these is a few lines.

Medium

- The lexer: `tokenize`, `read_annotation` and `read_keyword`. Draw your state diagram first, since it's the same machine.
- `parse_simple_item`: the shared choices for the body, paragraph and list item.
- `parse_bold`, `parse_list_items`, `parse_body`: the loops. Be able to say what ends each loop.

Hard

- `translate()`: this is the one you'll need to study.
  - It reads the `record` list of accepted tokens and walks it with the index `i`.
  - `vars`, `marks` and `closers` have to stay in step.
  - Each `i += 2`, `i += 4` or `i += 5` skips exactly the tokens that construct used.
- `resolve()`: it's short, but you need to be able to say why it searches `vars` from the end backward.

How to learn `translate()`

1. Take `Test8.lol` on paper.
2. Write out the `record` list the parser produces for it.
3. Walk the loop and write down `vars`, `marks` and `closers` after each token.
4. Once you've seen the second paragraph get `happiness` instead of `42`, the comments will come easily.

If you hit a function you can't explain, ask me about it. I'll explain what it does, and you write the comment.

but do u think each function is needed

Checking which helper functions can be dropped ▸

Not every one. The trait methods are required, and so is the core pipeline. A handful of helpers exist for convenience, and the code would work without them.

Required (the project or the design needs these)

- All 25 trait methods: `compile`, `next_token`, `parse`, `current_token`, `set_current_token`, `get_char`, `add_char`, `lookup`, and the 17 `parse_*`. Your instructor's tests call them.
- `tokenize`: the main lexer loop, as in your lab.
- `peek_token`: `#MAEK` and `#GIMMEH` each start several rules, so the parser needs to look one token further to choose.
- `expect`, `consume`, `is_text`, `syntax_error`: the basic parser machinery.
- `translate`, `resolve`: the scope check and the HTML output.

- `main` : usage checks, file I/O and launching the browser.

Convenience only (could be removed or folded in)

- `unexpected` and `found` : they only build error messages. You could write one `syntax_error(...)` call at each spot instead.
- `expect_word` : it checks for a single word without spaces. You could do that check inline in `parse_variable_define` and `parse_link` .
- `read_annotation` and `read_keyword` : you could put both inside `tokenize` , which makes `tokenize` long.
- `lexical_error` : it only prints and exits, so you could inline it.
- `emit` : it's two lines, and you could write `out.push_str(...)` directly.
- `flush_text` : it's used in two places, so keep it.
- `parse_simple_item` : without it you'd copy the same six-way choice into `parse_body` , `parse_inner_text` and `parse_inner_list` . It saves lines and bugs.
- `open_in_browser` : you could put it in `main` .

Removing the convenience functions saves roughly 40-60 lines, but the code gets harder to read, because you repeat messages and checks. I'd keep them, since each does one clear job and is easy to comment.

I can fold in any of these if you want. Tell me which ones, and I'll re-run all the tests afterward.

when I looked over my prjkect does my parser build AST I dont see it

Tracing how nesting is implied by the tag order ▸

You're right that there's no tree in the code. What the parser builds is a flat list.

What it does

- Every time a parse function accepts a token, `consume()` pushes it onto `record` , and its line number onto `record_lines` .
- By the end of parsing, `record` holds every accepted token in order. For Test8 that's `#HAI` , `#OBTW` , the comment text, `#TLDR` , `#MAEK` , `HEAD` , and so on.
- The nesting is still in that list. A paragraph's `#MAEK PARAGRAF` comes before everything inside it, and its `#MKAY` comes after.
- `translate()` recovers the nesting with the `closers` stack. It pushes the closing tag when a block opens and pops it at `#MKAY` or `#OIC` .

Does that satisfy the project?

The project text says the parser "builds an abstract syntax (i.e., parse) tree, conceptually",

and that it "may best be done using a stack(s) or an array list/vector(s)". Your design notes say to record the recognized structure "in a vector / array / stack / conceptual parse tree". A vector of accepted tokens fits that wording, and it's why `record` exists.

The limit
It is a token list, not a tree you could draw with parent and child nodes. If your instructor expects real nodes, this would fall short. I can't tell from the project text which he wants.

If you want a real tree
I could add a node struct with a kind, some text and a `Vec` of child nodes, have each `parse_*` function build and return its node, and make `translate()` a recursive walk over the tree. It uses only structs, vectors and recursion, which your class covered. It would add roughly 60-100 lines and mean rewriting the parser and `translate()`, so I'd re-run all the tests.

If you're unsure, ask your instructor whether a vector of accepted tokens counts as the parse tree. That one question settles it.

so did I satisfy it or not

Probably yes, but I can't be certain.

The project says the parser builds the tree "conceptually" and that it's best done with "a stack(s) or an array list/vector(s)". Your design notes also list a vector of recognized structure as fine. `record` is a vector of everything the parser accepted, in order, and `translate()` rebuilds the nesting from it. That fits the wording.

The risk is that a grader expects real tree nodes with children. The project doesn't say that, so I think the risk is small. If you want to be sure, send your instructor a one-line question: "Is a vector of accepted tokens enough for the parse tree?"

If he says no, I can convert it to a node tree.

final verdict do I rewerite it yhes or no

Weighing whether a rewrite is worth the risk.
No, don't rewrite.

- **It matches the wording.** The project says the tree is built "conceptually", and that a stack or vector is the best way to do it. Your design notes list a vector as acceptable too.
- **Tests are 20 points.** Those points depend on the output, and that's already working.

- **You're out of time.** The deadline is tomorrow night, and a rewrite risks new bugs in code that passes everything. You still need to comment the code, fill in the readme and design files, build on your machine and push.

Cheap insurance

- **Comment `record` yourself:** in your own words, say that it is the parse tree, stored as a flat vector in the order the parser accepts tokens, and that `translate()` rebuilds the nesting from it.
- **Ask your instructor** if you can reach him before the deadline.

If he comes back saying it needs real nodes, you'd have no time left, so that's the only real risk. The wording of the project makes it unlikely.

where is ectra test casdes

The extra test cases are in the `mine` folder, inside the `test` folder next to the instructor's files:

```
your-repo/
└── test/
    ├── Test1.txt ... Test10.lol     (your instructor's)
    └── mine/                        (my 24 extra tests)
```

I created them in this chat. If you copied only `main.rs` and `Cargo.toml`, you won't have them yet, so you need to download them from this conversation.

What's in `mine/`

- **Valid files (`ok_*`):** `ok_01_list_newline_link`, `ok_02_vars_in_bold_and_items`, `ok_03_item_scope`, `ok_04_three_level_scope`, `ok_05_comments_no_head_mixed_case` and `ok_06_title_only_head_then_global_var`. These should compile.
- **Syntax errors (`err_syntax_01` to `err_syntax_13`):** things like a missing `#KBYE`, a nested paragraph, a second head and text after `#KBYE`. These should fail.
- **Lexical errors (`err_lexical_01` to `err_lexical_03`):** an unknown annotation, a bad keyword and a lone `#`. These should fail.
- **Semantic errors (`err_semantic_01` to `err_semantic_03`):** a variable used after its scope ends, a variable that leaks out of a list item, and a variable defined only in a later block. These should fail.